

Common challenges in high level system design

Availability: operational/accessible -in 9's SLA's

Fault tolerant: Mumbai floods, all the data centres in Mumbai will go down

Low latency:

Time taken between req sent and response received

----->

<-----

If user is in India and server is in US, latency is more.

Inorder to reduce latency, we need to store backend code/service running somewhere near to us

### **AWS Regions:**

Are separate geographical areas, there are multiple data centres in each geographical area

Within region, there are multiple availability zones

Each AZ with its own redundant power and network - this is availability zone.

AWS-Region and availability zones.

AWS VPCs

<https://www.kerno.io/blog/what-is-aws-vpc-tutorial>

<https://www.kerno.io/aws/ec2>

-----

Node.js

Javascript engine which can help .js run outside the browser

**Express.js (or simply Express)** is a **web framework for Node.js** that helps you build web servers and APIs easily.

xpress is a lightweight library that makes it easier to handle:

- HTTP requests (GET, POST, etc.)
- Routes (URLs like /home, /api/users)
- Middleware (logic that runs between request and response)

## **Why do we need Express?**

Without Express (using plain Node.js), creating a server is more complex:

```
const http = require('http');
```

```
http.createServer((req, res) => {  
  if (req.url === '/') {  
    res.end('Hello World');
```

```
}  
}).listen(3000);
```

Works, but:

- Routing is manual
- Code gets messy as the app grows

With Express (much simpler):

```
const express = require('express');  
const app = express();
```

```
app.get('/', (req, res) => {  
  res.send('Hello World');  
});
```

```
app.listen(3000);
```

Cleaner, easier, scalable.

Npm - node package manager

Difference between module and a package

Package - is a combination of one or more modules: eg: configuration, documentation,

Package managers

Npm - node

Pip - python

Maven/gradle - java

Express - web app framework based on node

Request handling , http handling easy

Express package is available in rpm, which can be installed

---

Basic express app:

npm init -y

**What this does:**

- Creates a `package.json` file
- This file tracks:

- Project name
- Version
- Dependencies (like Express)
- Scripts (commands you can run)

The `-y` flag skips all questions and uses defaults.

## Step 2: Install Express

`npm install express`

### What this does:

- Downloads Express into `node_modules`
- Adds it to `package.json` under `dependencies`

## Step 3: Create your app file

Create a file called:

`app.js`

```
const express = require('express');
```

```
const app = express();
```

```
// Define a route
```

```
app.get('/', (req, res) => {
  res.send('Hello, World!');
});
```

```
// Start server
```

```
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});
```

```
node app.js
```

`http://localhost:3000`

Hello, World!

```
const express = require('express')
```

- Imports the Express library

```
const app = express()
```

- Creates an **Express application**

- Think of it as your server object

**app.get( '/', handler )**

- Defines a **route**
- / → homepage
- Runs the handler when user visits it

(req, res) => {}

Show more lines

- req → incoming request (data from client)
- res → response you send back

**res.send( 'Hello, World!' )**

- Sends data back to the browser

**5. app.listen( PORT )**

- Starts the server
- Listens for incoming requests on that port

## Optional: Add a Script (Cleaner execution)

Edit package.json:

```
"scripts": {  
  "start": "node app.js"  
}  
npm start
```

Example: Add another route

```
app.get('/about', (req, res) => {  
  res.send('This is the About page');  
});
```

http://localhost:3000/about

| Concept      | Meaning                  |
|--------------|--------------------------|
| npm init     | Create project config    |
| express      | Web server framework     |
| app.get()    | Handle HTTP GET requests |
| req          | Request data             |
| res          | Response object          |
| app.listen() | Start server             |

AWS:

Elastic IP: does not change if service restarts

---

Loadbalancing between 2 EC2 instances

Below is a **clear end-to-end walkthrough** to deploy your Express app on **2 EC2 instances behind a Load Balancer**, including all AWS networking concepts you mentioned.



## Architecture Overview

Internet



Internet Gateway (IGW)



Public Subnet (AZ1) — EC2 Instance 1 (Express app)



Public Subnet (AZ2) — EC2 Instance 2 (Express app)



Application Load Balancer (ALB)



## Step 1: Create a VPC

Go to:

**AWS Console → VPC → Create VPC**

- Name: my-vpc
- CIDR block: 10.0.0.0/16

👉 This is your private network.

## ✅ Step 2: Create Subnets (2 AZs)

Create 2 public subnets:

| Subnet   | AZ          | CIDR        |
|----------|-------------|-------------|
| subnet-1 | ap-south-1a | 10.0.1.0/24 |
| subnet-2 | ap-south-1b | 10.0.2.0/24 |

✅ Why:

- High availability
- Load balancer needs multiple AZs

## ✅ Step 3: Attach Internet Gateway (IGW)

Create IGW:

- Name: my-igw

**Attach to VPC**

👉 Enables internet access

## ✅ Step 4: Create Route Table

Create:

- Name: public-rt

### **Add route:**

0.0.0.0/0 → Internet Gateway

### **Associate:**

- subnet-1
- subnet-2



Now both subnets are **public**



## **Step 5: Security Groups**



### **EC2 Security Group**

Allow:

HTTP (80) → 0.0.0.0/0

Custom TCP 3000 → ALB SG (later)

SSH (22) → Your IP only



### **Load Balancer Security Group**

Allow:

HTTP (80) → 0.0.0.0/0



## **Step 6: Launch 2 EC2 Instances**

### **Go to:**

EC2 → Launch Instance

### **Config:**

- AMI: Amazon Linux 2
- Instance type: t2.micro
- Network: your VPC

- Subnet:
  - Instance 1 → subnet-1
  - Instance 2 → subnet-2
- Auto-assign public IP 
- Security group: EC2 SG

## Step 7: Install Node.js + App

SSH into each instance:

Shell

```
ssh ec2-user@<public-ip>
```

Show more lines

### Install Node.js

Shell

```
sudo yum update -y
sudo yum install -y nodejs npm
```

Show more lines

### Create app

Shell

```
mkdir app
cd app
npm init -y
npm install express
```

Show more lines

### Create file:



Shell

```
nano app.js
```

Show more lines

Paste your code:

JavaScript

```
const express = require('express');
```

```
const app = express();
```

```
app.get('/', (req, res) => {  
  res.send('Hello, World from EC2!');  
});
```

```
const PORT = 3000;  
app.listen(PORT, '0.0.0.0', () => {  
  console.log(`Server running on port ${PORT}`);  
});
```

Show more lines

## Run app

Shell

```
node app.js
```

```
``
```

Show more lines



Important:

```
'0.0.0.0'
```

→ allows external access

## ✅ Step 8: Test EC2 directly

Open browser:

`http://<EC2-public-IP>:3000`

## ✅ Step 9: Create Load Balancer

**Go to:**

EC2 → Load Balancers → Create

Choose: 👉 **Application Load Balancer**

### **Configuration:**

- Name: `my-alb`
- Scheme: Internet-facing
- VPC: your VPC
- Subnets:
  - subnet-1
  - subnet-2

### **Security Group:**

Use ALB SG

## ✅ Step 10: Create Target Group

- Type: Instances
- Protocol: HTTP
- Port: **3000**
- Health check path: /

Register:

- EC2 instance 1
- EC2 instance 2

EC2 service has the /health implemented

```
const os = require('os');
const express = require('express');

const app = express();

// Get server IP
let serverIPAddress = 'unknown';

const interfaces = os.networkInterfaces();
for (const name of Object.keys(interfaces)) {
  for (const iface of interfaces[name]) {
    if (!iface.internal && iface.family === 'IPv4') {
      serverIPAddress = iface.address;
    }
  }
}

// Root route
app.get('/', (req, res) => {
  res.send(`Hello from server ${serverIPAddress}`);
});

//  Health check endpoint (for ALB)
app.get('/health', (req, res) => {
  res.status(200).json({
    status: 'ok',
    server: serverIPAddress
  });
});

// Other routes
app.get('/user', (req, res) => {
  res.status(200).send(`User API from ${serverIPAddress}`);
});

app.get('/payment', (req, res) => {
  res.status(200).send(`Payment API from ${serverIPAddress}`);
});

// Start server
const PORT = process.env.PORT || 3000;

app.listen(PORT, '0.0.0.0', () => {
```

```
console.log(` Server running on port ${PORT}`);
});
```

## ✅ Step 11: Configure Listener

- Listener: HTTP : 80
- Forward → target group

## ✅ Step 12: Access Load Balancer

After creation:

```
http://<ALB-DNS>
```

✅ You'll see:

```
Hello, World from EC2!
```

## ✅ Step 13: Verify Load Balancing

Modify apps:

**Instance 1**JavaScript

```
res.send('Hello from Instance 1');
```

Show more lines

**Instance 2**

JavaScript

```
res.send('Hello from Instance 2');
```

**Refresh browser multiple times:**

👉 You'll see responses alternating

✓ Load balancing confirmed



## Concepts Explained



### VPC

Virtual private network in AWS



### AZ (Availability Zone)

- Independent data centers
- Improves reliability



### Public Subnet

- Has route to internet via IGW



### Internet Gateway

- Allows internet access



### NAT Gateway (Not needed here)

- Used for **private subnets**
- For outbound internet only



### Security Groups

- Stateful firewall
- Control traffic at instance level

## ✓ Route Table

- Controls traffic routing
- 0.0.0.0/0 → IGW = internet access

## ✓ Load Balancer

- Distributes requests across instances
- Improves availability & scalability

## ✓ Final Architecture Summary

| Component   | Purpose           |
|-------------|-------------------|
| VPC         | Network           |
| Subnets     | Split across AZs  |
| IGW         | Internet access   |
| Route Table | Routing rules     |
| EC2         | Run app           |
| SG          | Security          |
| ALB         | Load distribution |

## Next Enhancements

- Use **Auto Scaling Group**
- Use **PM2** instead of node
- Add **HTTPS (SSL)**
- Move EC2 to **private subnet**

## What is a Listener?

A **listener** tells the Load Balancer:

👉 “When traffic comes in on this port and protocol, what should I do with it?”

## ✅ Your configuration:

Listener: HTTP : 80

Action: Forward → Target Group

## 🔍 Why this step is needed

### ✅ 1. Entry point for traffic

Users access your app like this:

`http://<ALB-DNS>`

👉 By default, that means:

- Protocol: HTTP
- Port: 80

So the **listener watches port 80** for incoming requests.

### ✅ 2. Without listener → nothing happens

Even if:

- EC2 instances are running ✅
- Target group is created ✅

❌ The Load Balancer won't know:

- Where to send traffic

👉 So requests will fail

### ✅ 3. It connects ALB → Target Group

This is the key flow:

```
Client → ALB (port 80)
      ↓
Listener checks rule
      ↓
Forwards request
      ↓
Target Group (port 3000)
      ↓
EC2 instances
```

### ✅ 4. Protocol & Port Mapping

Your app runs on:

EC2 → port 3000

But users access:

ALB → port 80

👉 Listener bridges this gap

### ✅ 5. Routing Rules (Basic vs Advanced)

For now:

All traffic → target group

But in real apps you can do:

/api → API servers

/images → image servers

👉 Listener enables routing logic



## Simple Analogy

Think of the Load Balancer as a **reception desk**:

- **Listener** = receptionist
- **Target group** = employees
- **Requests** = visitors

👉 Receptionist decides:

“Who should handle this request?”

## ✅ What happens if you skip this?

- ALB exists ✅
- EC2 instances exist ✅
- Target group exists ✅

❌ But:

- No traffic routing
- Browser will not get response

## ✅ Key Insight

| Component    | Role                              |
|--------------|-----------------------------------|
| Listener     | Accepts traffic + decides routing |
| Target Group | Holds EC2 instances               |
| EC2          | Processes request                 |

## ✅ Summary

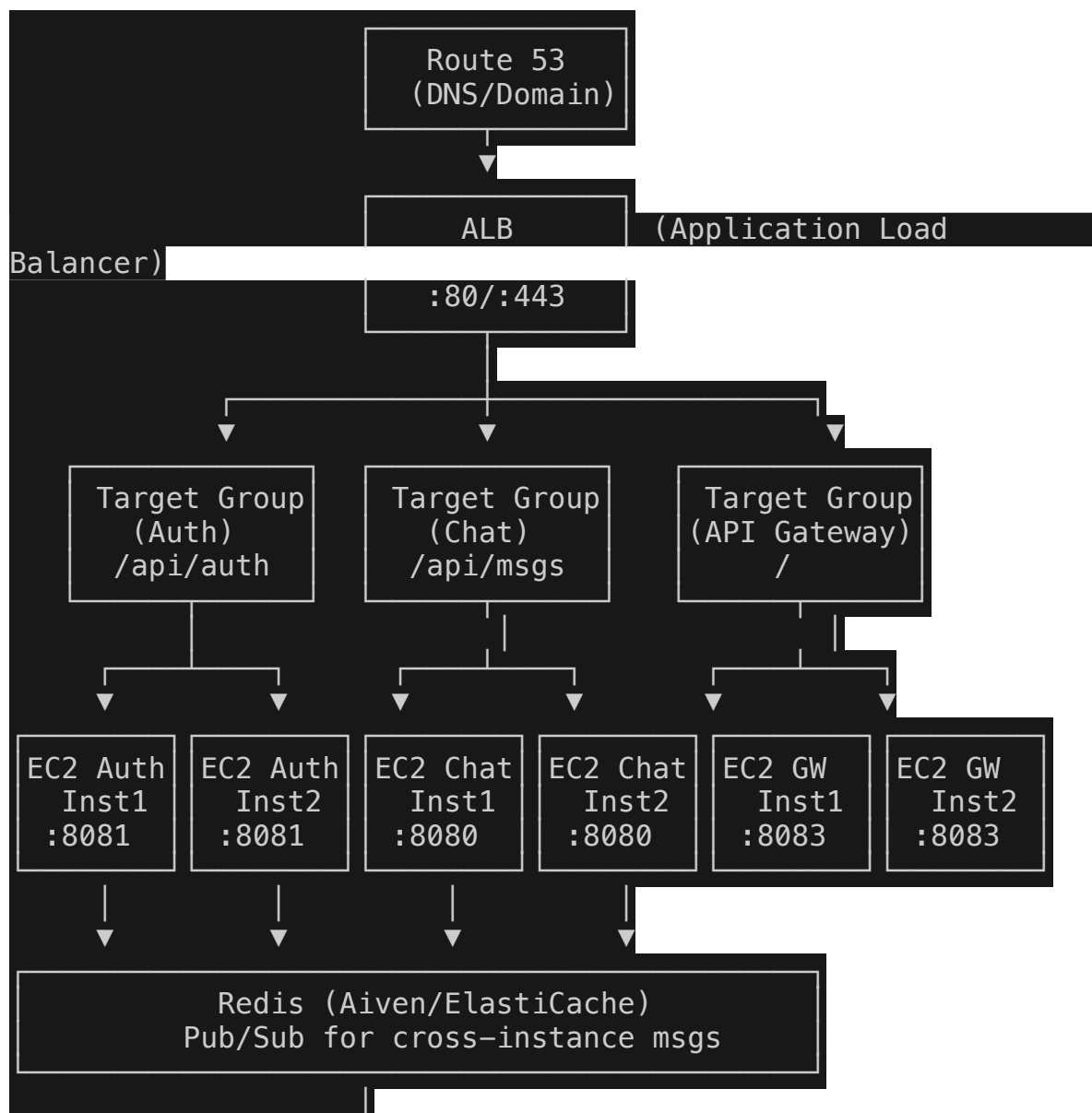
👉 You configure a listener because:

- It defines how the LB receives traffic (port 80)
- It links incoming traffic to your instances
- It enables load balancing to actually happen

Chat App

## Updated Architecture with ALB & Multiple EC2 Instances

AWS Architecture with Load Balancing



▼

MongoDB Atlas /  
AWS DocumentDB

## ALB Configuration

| Component       | Configuration                                                    |
|-----------------|------------------------------------------------------------------|
| ALB Listener    | Port 80 (HTTP) → redirect to 443; Port 443 (HTTPS) with ACM cert |
| Sticky Sessions | Enabled on Chat Target Group (required for WebSocket/Socket.io)  |
| Health Checks   | GET / on each service port                                       |

## Target Groups

| Target Group | Path Rule                     | Port | Instances       | Health Check |
|--------------|-------------------------------|------|-----------------|--------------|
| tg-auth      | /api/auth/*                   | 8081 | 2 EC2 (Auth)    | GET / → 200  |
| tg-chat      | /api/messages/*, /socket.io/* | 8080 | 2 EC2 (Chat)    | GET / → 200  |
| tg-gateway   | /* (default)                  | 8083 | 2 EC2 (Gateway) | GET / → 200  |

## ALB Listener Rules (Priority Order)

Rule 1: IF path = /api/auth/\* → FORWARD to tg-auth  
Rule 2: IF path = /api/messages/\* → FORWARD to tg-chat  
Rule 3: IF path = /socket.io/\* → FORWARD to tg-chat (sticky)  
Rule 4: DEFAULT → FORWARD to tg-gateway

## Key AWS Services

| Service | Purpose                                                     |
|---------|-------------------------------------------------------------|
| ALB     | Layer 7 load balancing, path-based routing, SSL termination |

|                                 |                                                                    |
|---------------------------------|--------------------------------------------------------------------|
| <b>Target Groups</b>            | Logical grouping of EC2 instances per microservice                 |
| <b>EC2 (Auto Scaling Group)</b> | Min 2, Max 4 instances per service for HA                          |
| <b>ElastiCache (Redis)</b>      | Managed Redis for Pub/Sub (replaces Aiven in AWS)                  |
| <b>DocumentDB</b>               | Managed MongoDB-compatible DB (replaces Atlas)                     |
| <b>ACM</b>                      | SSL/TLS certificates for HTTPS                                     |
| <b>Security Groups</b>          | ALB → allows 80/443 inbound; EC2 → allows traffic only from ALB SG |

WebSocket Handling with ALB

Since Socket.io uses WebSockets, you need:

- 1 **Sticky sessions** (session affinity) on the chat target group
- 2 **Idle timeout** increased to 3600s on ALB for persistent connections
- 3 **Redis Pub/Sub** remains critical — even with sticky sessions, messages between users on different instances must route through Redis

Auto Scaling Policy

```

Auto Scaling Group
Min: 2 | Desired: 2 | Max: 4
Scale Out: CPU > 70% for 5 min
Scale In:  CPU < 30% for 10 min

```

Security Groups

```

ALB SG:      Inbound: 80, 443 from 0.0.0.0/0
EC2 SG:      Inbound: 8080-8083 from ALB SG only
Redis SG:    Inbound: 6379 from EC2 SG only
DocumentDB SG: Inbound: 27017 from EC2 SG only

```

Why This Design?

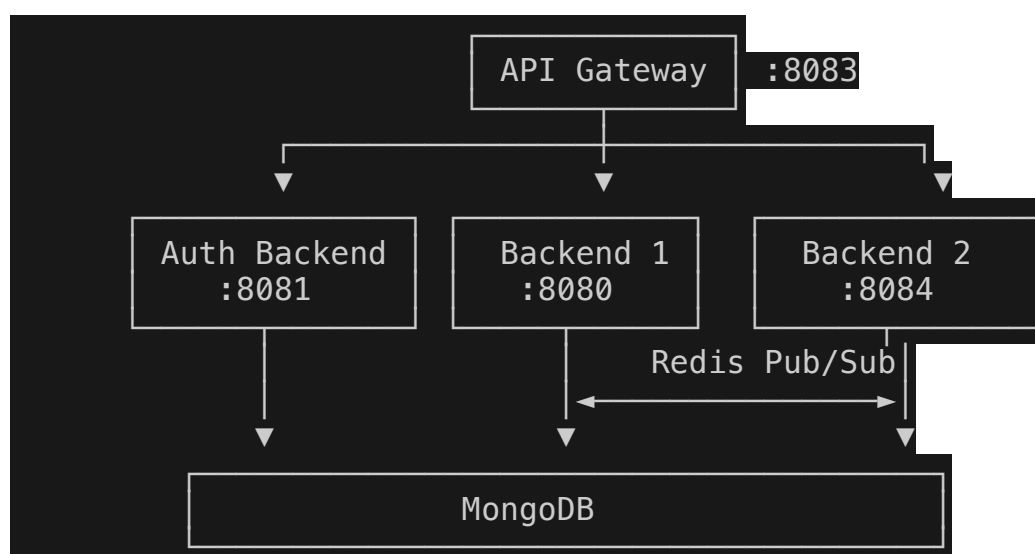
- **ALB path-based routing** eliminates the need for the Node.js API Gateway in production
- **Redis Pub/Sub** ensures real-time messages work across multiple chat server instances
- **Sticky sessions** keep a user's WebSocket connected to the same backend
- **Auto Scaling** handles traffic spikes automatically
- **Multi-AZ** deployment across target groups provides high availability

# Chat App – Full Architecture & AWS Deployment

## Tech Stack

| Layer            | Technology                                                           |
|------------------|----------------------------------------------------------------------|
| Frontend         | Next.js, React, Tailwind CSS, Socket.io-client, Zustand (state mgmt) |
| Chat Backend     | Node.js, Express, Socket.io, Mongoose                                |
| Auth Backend     | Node.js, Express, bcrypt, JWT                                        |
| API Gateway      | Node.js, http-proxy-middleware                                       |
| Database         | MongoDB (Atlas or DocumentDB)                                        |
| Pub/Sub          | Redis (Aiven) for cross-server messaging                             |
| Containerization | Docker, Docker Compose                                               |
| Cloud            | AWS EC2, ECS, ECR                                                    |

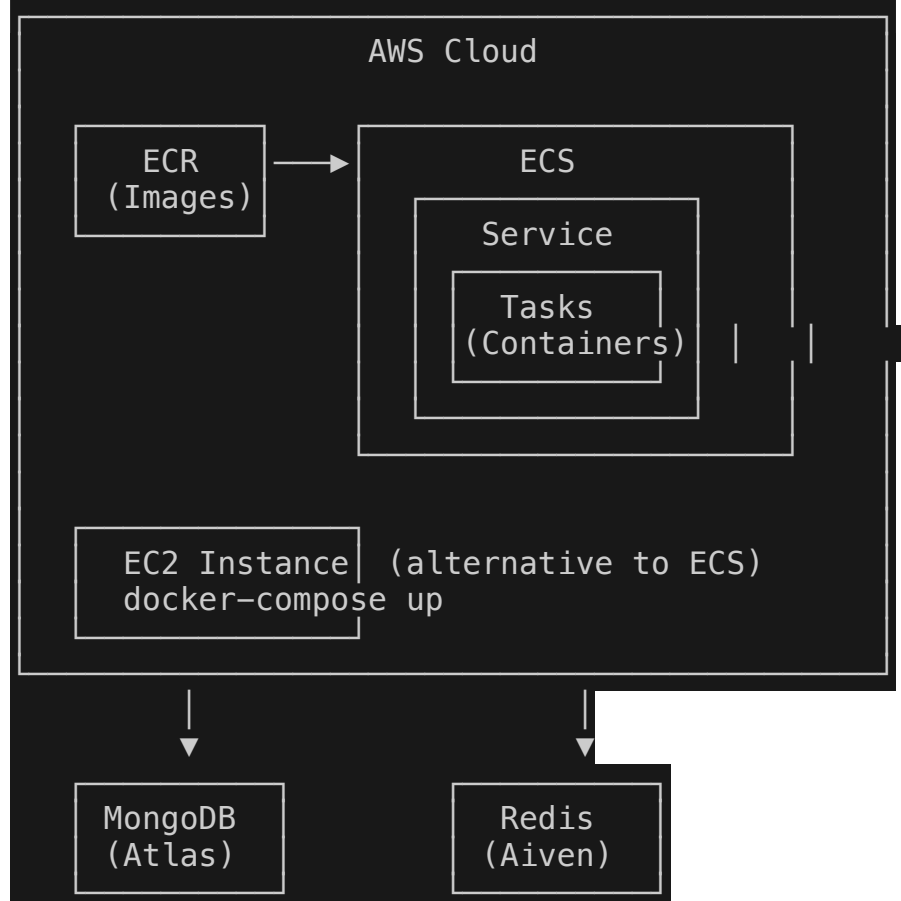
## High-Level Architecture



## 6-Step Build Process

| Step | Feature                       | Key Concepts                                                         |
|------|-------------------------------|----------------------------------------------------------------------|
| 1    | Basic 1-server chat           | WebSockets (Socket.io), 1:1 messaging                                |
| 2    | Multi-server support          | <b>Redis Pub/Sub</b> to route msgs across backends via userSocketMap |
| 3    | Group messaging               | Extending conversation model                                         |
| 4    | Offline messaging             | Store msgs in MongoDB, fetch on reconnect                            |
| 5    | Authentication                | JWT cookies, bcrypt, middleware (verifyToken)                        |
| 6    | Containerization & Deployment | Docker, Docker Compose, AWS                                          |

## AWS Deployment Architecture



## AWS Services Used

| Service | Purpose |
|---------|---------|
|---------|---------|

- EC2** Host Docker containers directly via docker-compose up
- ECS** Managed container orchestration (Services → Task Definitions → Tasks)
- ECR** Private Docker image registry (like DockerHub)

#### Key Design Patterns

- **Microservices** – Auth, Chat, API Gateway as separate services
- **Pub/Sub (Redis)** – Enables multi-server real-time messaging; if receiver is on a different backend, message is published to Redis channel chat\_{username}
- **User-Socket Map** – In-memory map linking usernames to socket connections per server
- **Stateless Auth** – JWT in HTTP-only cookies, verified via middleware
- **API Gateway** – Single entry point proxying /api/auth, /api/users, /api/msgs to respective backends

## Chat App – Architecture & Design Overview

### Microservices Architecture

The project is split into **two backend services**:

```
Chat-App-main/
├── auth-backend/      # Authentication microservice
├── backend/           # Chat/messaging microservice
└── (frontend/)       # Likely a frontend client
```

### Components

| Component           | Responsibility                                        |
|---------------------|-------------------------------------------------------|
| <b>auth-backend</b> | User signup, login, JWT token generation              |
| <b>backend</b>      | Chat/messaging logic, likely real-time via WebSockets |
| <b>MongoDB</b>      | Database for users, messages, conversations           |
| <b>Docker</b>       | Containerization (.dockerignore present)              |

### Tech Stack

- **Runtime:** Node.js with ES Modules (import/export)
- **Framework:** Express.js

- **Database:** MongoDB via **Mongoose ODM**
- **Auth:** JWT tokens stored as HTTP cookies + **bcrypt** password hashing
- **Containerization:** Docker
- **Real-time:** Likely **Socket.io** (standard for Node.js chat apps)

### High-Level Design



### Auth Flow

- 1 **Signup/Login** → Password hashed with bcrypt → JWT generated → Set as **HTTP cookie**
- 2 Subsequent requests carry the JWT cookie for **stateless authentication**
- 3 The chat backend validates the JWT to authorize messaging

### Key Design Patterns

- **Microservices** – Auth and chat logic are decoupled into separate services
- **Stateless Auth** – JWT cookies, no server-side sessions
- **MVC Pattern** – Controllers (auth.controller.js), Models (user.model.js), Utils (generateToken.js)
- **Containerized Deployment** – Each service has its own Docker config

### Minor Issue in Code

The signup route returns **201** even when the username already exists — it should return **409 (Conflict)** instead.

## AWS Constructs & Databases

### AWS Constructs

**None visible.** Based on the codebase, this project does **not** use any AWS services. It uses:

- **Docker** for containerization (could be deployed anywhere — AWS ECS/EKS, but no AWS SDK or config is present)
- Standard Node.js/Express servers



## Database Used

| Database                                                                                                                                                                                                                                                                                                                                                                                                               | Why                                 |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------|
| <b>MongoDB</b> (via Mongoose ODM)                                                                                                                                                                                                                                                                                                                                                                                      | Used for storing users and messages |
| <b>Why MongoDB?</b>                                                                                                                                                                                                                                                                                                                                                                                                    |                                     |
| <ul style="list-style-type: none"><li>• <b>Schema flexibility</b> – Chat messages can vary in structure (text, media, metadata)</li><li>• <b>Fast reads/writes</b> – Suited for real-time chat with high throughput</li><li>• <b>Document model</b> – Natural fit for conversations (nested messages, user references)</li><li>• <b>Easy scaling</b> – Horizontal scaling via sharding for growing chat data</li></ul> |                                     |
| If Deployed on AWS, Likely Services Would Be:                                                                                                                                                                                                                                                                                                                                                                          |                                     |

| AWS Service                                                                                                                                         | Purpose                                       |
|-----------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| <b>ECS/EKS</b>                                                                                                                                      | Run Docker containers (auth-backend, backend) |
| <b>DocumentDB or MongoDB Atlas</b>                                                                                                                  | Managed MongoDB                               |
| <b>ALB</b>                                                                                                                                          | Load balancer in front of services            |
| <b>ECR</b>                                                                                                                                          | Store Docker images                           |
| But as of now, the project has <b>no direct AWS dependencies</b> — it's a Dockerized Node.js app with MongoDB that can run locally or on any cloud. |                                               |

-----  
-----

<https://docs.aws.amazon.com/vpc/latest/userguide/nat-gateway-scenarios.html>  
<https://docs.aws.amazon.com/vpc/latest/userguide/vpc-example-private-subnets-nat.html>  
<https://dev.to/aws-builders/aws-networking-aws-vpc-subnets-security-groups-nat-gateway-ip-addresses-4p8c>

-----  
-----

## Kubernetes

<https://portworx.com/blog/what-is-a-kubernetes-cluster-and-what-are->

kubernetes-master-nodes-api-servers-roles-and-rolebinding/

<https://medium.com/@chamathupeka/kubernetes-basics-pods-deployments-and-services-bab57fd1ba6f>

<https://www.acte.in/kubernetes-explained-features-benefits>